

Redesign dei form di **Anagrafica**

Analisi UI/UX del modulo **anagrafica** e di tutti i suoi sottomoduli — dipendenti, formazione, qualifiche/skill-matrix, sicurezza/visite mediche, organizzazione — con proposta di un **design system unico dei form**, mockup prima/dopo, alternative a confronto e piano a fasi.

Brief operativo di redesign

Vincolo: Django SSR + HTMX

Priorità: coerenza · velocità · accessibilità

PERIMETRO

103

template
ispezionati

CLUSTER

5

sottomoduli

SISTEMI CSS

~20

namespace
paralleli

ERRORI INLINE

≈0%

copertura nei
form

01 Sintesi esecutiva

Cosa abbiamo trovato, l'opportunità, la direzione consigliata

Il modulo **anagrafica** è funzionalmente ricco e ben pensato, ma la sua interfaccia è cresciuta per stratificazione: ogni pagina e ogni sottomodulo ha reinventato gli stessi mattoni — campo, bottone, pill di stato, modale, tabella. Oggi convivono circa **20 sistemi di stile paralleli**, e i form più usati aggirano i widget Django scrivendo HTML a mano, perdendo aiuto contestuale, marcatori di obbligatorietà coerenti e **gli errori di validazione accanto al campo**.

~20

NAMESPACE CSS CHE
DUPLICANO GLI STESSI
COMPONENTI

7_x

IL BOTTONE È
REIMPLEMENTATO ALMENO
7 VOLTE

≈0%

FORM CON ERRORI
MOSTRATI ACCANTO AL
CAMPO

3

MODI DIVERSI PER
"SCEGLI UN DIPENDENTE"

LA DIREZIONE, IN UNA FRASE

Esiste già la soluzione dentro il codice: il sistema fmd- della Formazione è maturo, usa la palette NOVICROM e gestisce correttamente campo, obbligatorio, help, **errori inline**, modale e pill di stato. La proposta è **promuoverlo a design system unico del portale**, con un solo renderer di campo, una sola pill, una sola modale e la validazione inline via HTMX — restando dentro il vincolo Django SSR + HTMX.

! I tre problemi che pesano di più

- ▶ **Coerenza.** Stesso mattone, 20 implementazioni. La UI cambia faccia tra una pagina e l'altra e la manutenzione costa moltissimo.
- ▶ **Velocità d'inserimento.** Form a mano senza autofill/validazione inline, select senza ricerca, "tieni premuto Ctrl", ricarica pagina per ogni riga.
- ▶ **Accessibilità ed errori.** Errori solo in cima (flash), label non associate, header cliccabili non da tastiera, azioni distruttive con `confirm()` nudo.

★ Cosa preservare (funziona già)

- ✓ Il sistema fmd-: sprite SVG, variabili CSS, form sezionati, fmd-autofill.
- ✓ I renderer di campo `_fmd_field.html` / `_fm_form_fields.html` (label, obbligatorio, help, errori).
- ✓ La ricerca live HTMX, il player e-learning, il partial candidati.
- ✓ I fogli di stampa separati (`@media print`) per attestati/registri/verbalì.

ESITO ATTESO

Un modulo che si presenta come **un solo prodotto**: form che si compilano più in fretta e spiegano gli errori dove serve, un'unica libreria di componenti da mantenere, e una base pronta per il dark-mode e l'accessibilità su tutte le pagine — a parità di stack e senza riscritture rischiose.

02 Metodo & perimetro

Cosa è stato ispezionato e con quali criteri

L'analisi ha coperto l'intero modulo anagrafica: **103 template** (pagine, partial e componenti) raggruppati in **5 cluster** funzionali, con lettura del markup, dei sistemi CSS, dei widget Django/HTML e delle interazioni HTMX/JS. L'attenzione è stata posta sui **form di inserimento, modifica e gestione**, come richiesto.

I 5 cluster

- ▶ **Dipendenti core** — creazione, scheda/edit, elenchi, documenti, retribuzioni, stampa.
- ▶ **Formazione** — corsi, sessioni, iscritti, presenze, e-learning, attestati.
- ▶ **Qualifiche / Skill Matrix** — qualifiche, sessioni rinnovo, matrice, mansioni.
- ▶ **Sicurezza / Visite / DPI** — rischi, wizard, visite mediche, verbali, conformità.
- ▶ **Organizzazione** — aree, ruoli, onboarding, import retribuzioni/cedolini, impostazioni.

Criteri di valutazione

- ▶ **Coerenza** — quanti sistemi visivi per lo stesso componente.
- ▶ **Correttezza form** — widget Django vs HTML a mano; obbligatorio; errori.
- ▶ **Velocità** — attriti d'inserimento, autofill, ricerca, ricariche.
- ▶ **Accessibilità** — label/for, focus, tastiera, aria, semantica bottoni.
- ▶ **Robustezza** — bug concreti già presenti nel markup.

LE TUE PRIORITÀ, RECEPITE

Il brief è tarato sulle tre priorità indicate: **accessibilità e chiarezza degli errori, velocità di inserimento dati e coerenza tra i form**. Il vincolo tecnologico resta **Django SSR + HTMX**; nella sezione 06 sono comunque messe a confronto anche opzioni più spinte, come richiesto.

03 Diagnosi

La frammentazione, i debiti trasversali e i bug già presenti

3.1 La frammentazione visiva

Nel solo cluster "Dipendenti core" convivono **8 namespace CSS**; nelle Qualifiche 7, nella Sicurezza altrettanti, nella Formazione tre famiglie parallele. Sono, in gran parte, reimplementazioni degli stessi mattoni.



1
design system
unico del portale

~20 prefissi CSS che duplicano bottone, campo, pill, modale, tabella, avatar, empty-state.

3.2 I dieci debiti trasversali

#	DEBITO	PRIORITÀ	EVIDENZA (FILE:RIGA)
1	~20 sistemi CSS paralleli per gli stessi componenti; bottone reimplementato 7+ volte.	Coerenza	dc/dp/hr/fmd/fm/ana/dr/pb · qs/ro/mc/skm/rf
2	Widget Django aggirati con HTML a mano → niente help, obbligatorio, errori.	Errori	dipendente_create.html:132-198 · forms.py:69-79
3	Renderer di campo riusabili esistono ma sono inutilizzati fuori dalla Formazione.	Coerenza	_fmd_field.html · _fm_form_fields.html
4	Marcatore "obbligatorio" in 4+ forme (badge, asterisco arancio, rosso, testo grezzo, nulla).	Accessib.	fmd-req #ff8a1f · fm-req #dc2626 · create:133
5	Errori di validazione solo in cima (flash), mai accanto al campo. In interi cluster: 0 <code>field.errors</code> .	Errori	cluster qualifiche: grep aria-invalid = 0
6	Semaforo "scaduto/in scadenza/valido" reimplementato 5+ volte con hex diversi, mentre <code>fm-pill-*</code> esiste.	Coerenza	detail:1669 · 1970 · 1865 · cr-/cf-/wz-
7	Modali reinventate inline ~10 volte ; niente <code>role=dialog</code> , focus-trap, ESC (salvo un caso).	Accessib.	iscritti:407 · istruttori:111 · sess_detail:297
8	Form di modifica costruiti in JS <code>innerHTML</code> : label senza <code>for</code> , rischio escaping/XSS.	Errori	iscritti:502 · rischi_fattori_list:193,207
9	Icone: sprite SVG vs emoji mescolate anche nello stesso cluster.	Coerenza	fmd <code><use #i-*></code> vs 🧑 📄 🔄 nelle hr/fm
10	"Scegli dipendente" in 3 modi : chips+picker, select piatta, <i>input numerico legacy-id a mano</i> .	Velocità	qual_sess_create:133 · iscritti:117 · skm_refresh:102

Priorità = mappatura sul problema che il debito colpisce di più. Severità cromatica: rosso = alto impatto, arancio = medio, giallo = contenuto.

3.3 Bug concreti già nel markup — quick win

BUG	EFFETTO	EVIDENZA
<code>hr-btn-danger</code> non è definito da nessuna parte	I pulsanti "Elimina" non appaiono rossi — azione distruttiva non riconoscibile.	<code>rischi_categorie:79</code> · <code>fattori:92</code> · <code>esposizioni:77</code>
<code>verbale_dpi</code> usa <code>hr-btn</code> senza includere il CSS	Bottoni toolbar completamente non stilizzati .	<code>verbale_dpi.html:44-45</code>
Selettori dark-mode che retroagiscono sul tema chiaro	Testi con colore "dark" anche in light .	<code>qual_sess_create:50,53,58</code>
Argomenti JS numerici non quotati (valore <code>None</code>)	Con valore vuoto la chiamata diventa JS invalido .	<code>qualifiche_list:139</code> · <code>rischi_fattori:85</code>
Paginatore che perde i filtri	Cambiando pagina si azzerano area/tipologia contratto.	<code>dipendenti_list:103-105</code> · <code>ex_dipendenti:129</code>
Doppia navigazione	Il subnav non fa override del blocco → due menu sovrapposti .	<code>visite_mediche_nuova_sessione:10</code>
Cache-busting CSS divergente (<code>?v=2</code> vs <code>?v=3</code>)	Rischio di CSS stale su parte delle pagine.	<code>dashboard/corsi ?v=2</code> · <code>corso_form ?v=3</code>
Classi bottone in conflitto (<code>primary</code> + <code>ghost</code>)	Stile ambiguo del pulsante.	<code>formazione_piano_detail:42-45</code>

PERCHÉ CONTANO ORA

Sono difetti a **basso costo e alto impatto**: si chiudono in Fase 1 (sezione 07) e restituiscono subito una UI più affidabile, indipendentemente dal redesign più ampio.

04 Analisi per cluster

Il form più critico e i debiti chiave, sottomodulo per sottomodulo

1 Dipendenti core

8 namespace

Form critico: `dipendente_create.html` (card collassabili con progressività obbligatori/facoltativi — buona idea) e la scheda `dipendente_detail.html` (**2661 righe**, tutte le card sempre nel DOM, tab solo lato client).

- ▶ La sezione "Dati account" scrive input a mano e **ignora i widget** di `DipendenteLegacyForm`; l'asterisco compare solo su "Nome", gli errori solo su alcuni campi.
- ▶ I form "aggiungi" della scheda (qualifica, visita, documento, contratto) e tutta la retribuzioni sono **HTML grezzo senza ModelForm** → nessun errore di campo.
- ▶ **Tre UI diverse** per "elenco dipendenti" (`fmd-table`, `hr-table`, `ana-table`) con **due librerie di sorting**.
- ▶ Funzione "aggiungi figlio" **duplicata** tra create e detail; header di card come `<div onclick>` non da tastiera.

2 Formazione

3 famiglie

Form critico: `formazione_corso_form.html` è l'esempio migliore del modulo (5 fieldset semantici, assist `fmd-autofill`) — ma `sessione_form`, `iscritti` e `presenze` sono molto più poveri.

- ▶ Convivono `fmd-` (nuovo, curato) e `fm-/hr-` (vecchio); una pagina è dichiaratamente ibrida (`class="fmd hr-shell"`).
- ▶ La **stessa** modifica (corso, sessione) ha **due UI**: pagina `fmd-` e modale `fm-/hr-`.
- ▶ `iscritti/presenze`: campi a mano, select dipendente **senza ricerca**, **ricarica pagina per ogni riga**; modali con campi in `innerHTML`.
- ▶ Due pagine "Impostazioni" dello stesso modulo con **linguaggi opposti** (asterisco arancio vs rosso).

3 Qualifiche / Skill Matrix / Mansioni

7 namespace

Form critico: `qualifica_sessione_create.html` (create collettivo con tabella candidati HTMX) e la griglia `skm_match_validazione.html`.

- ▶ **Nessun form usa i Django Form**: zero `field.errors/aria-invalid` nell'intero cluster; errori solo in cima.
- ▶ Il prefisso `qs-` è **ridefinito 3 volte** con valori divergenti (copia-incolla).
- ▶ Il submit del create **non si disabilita a 0 partecipanti** → sessione vuota inviabile.
- ▶ `skm_refresh` chiede di **digitare a mano il "legacy id"** del dipendente — la peggiore delle tre UI di scelta persona.
- ▶ "Matrice competenze" è in realtà **read-only**: naming e aspettativa disallineati.

Form critico: `visite_mediche_nuova_sessione.html` — il wizard operativamente più importante del cluster, ma **interamente inline**, fuori da ogni design system.

- ▶ **Tre+ sistemi di semaforo** (`cr-`, `cf-`, `wz-badge`, oltre a `fm-pill/hr-pill`) con verdi che non coincidono nemmeno come hex.
- ▶ Il `_safety_subnav` ha **voci irraggiungibili**: categorie/fattori/esposizioni non si evidenziano né si raggiungono.
- ▶ Form **create (widget) vs edit (JS innerHTML)** divergenti nelle stesse pagine rischi, con iniezione grezza nei `<textarea>`.
- ▶ La ricerca dipendente del wizard usa `fetch()`+DOM manuale invece di HTMX come il resto del cluster.

Form critico: gli **import** (`retribuzioni_import`, con i gemelli `cedolini/contratti`) e `aree_list` (liste con creazione a tab e modifica a modale).

- ▶ Di nuovo **un namespace per pagina**: `ri-` (`import`), `cat-` (`aree/reparti`), con bottoni, pill e badge propri.
- ▶ `retribuzioni_import` **reimplementa i flash message inline** invece di `flash_messages.html`; ha una *upload-zone drag&drop* a mano ben fatta — ma bespoke — e un badge di stato tutto suo.
- ▶ `aree_list`: create con input a mano e asterisco **rosso #ef4444 — una terza tonalità** di "obbligatorio"; nessun errore di campo.
- ▶ Modifica via modale popolata da `data-*` (meglio dell'innerHTML, ma inline, senza ESC/focus-trap); URL edit via `replace('/0/'...)`.
- ▶ `.hr-tabs` **ridefinita localmente** con un aspetto diverso dal componente condiviso → collisione di nome; il caporeparto è ancora una `<select>` piatta senza ricerca.
- ▶ `cat-btn-danger` è definito qui, mentre altrove `hr-btn-danger` manca: il "rosso elimina" **dipende dalla pagina**.

05 La proposta: un design system unico dei form

Dentro Django SSR + HTMX — un solo set di componenti, riusati ovunque

Non serve inventare un linguaggio nuovo: bisogna **eleggerne uno**. Il sistema `fmd-` della Formazione è già maturo, usa la palette NOVICROM ed è l'unico che gestisce correttamente obbligatorio, help ed errori inline. La proposta è estrarlo in core come **design system del portale** e convergerci sopra tutti i form.

5.1 Cinque principi

- ▶ **Un renderer di campo, ovunque.** Un solo partial `_field.html` per label, obbligatorio, help ed errori — mai più input a mano.
- ▶ **Un solo sistema di stato.** Le `fm-pill-*` diventano la fonte unica; spariscono i semafori paralleli.
- ▶ **Una sola modale.** Componente unico con ESC, focus-trap e backdrop; niente più `innerHTML`.
- ▶ **Validazione dove serve.** HTMX valida il campo al blur e ridisegna l'errore accanto ad esso.
- ▶ **Accessibile per default.** Label associate, focus visibile, bottoni veri, aria — nel componente, non pagina per pagina.

5.2 I componenti canonici

COMPONENTE	FONTE UNICA PROPOSTA	SOSTITUISCE
Token & palette	variabili <code>--fmd-*</code> in <code>core</code>	hex sparsi in ogni pagina
Campo	<code>_field.html</code> (da <code>_fmd_field</code>)	input a mano · <code>_fm_form_fields</code>
Obbligatorio	<code>.nx-req</code> arancio	badge · <code>*</code> rosso/testo · nulla
Errore campo	<code>.nx-err</code> + <code>aria-invalid</code>	solo flash in cima
Pill di stato	<code>fm-pill-*</code>	<code>dp/hr/ana/cr/cf/wz-badge</code>
Bottone	<code>.nx-btn</code> (+ <code>-danger</code>)	7 set diversi · <code>danger</code> mancante
Modale	<code>fmd-modal</code> + JS unico	~10 modali inline
Scelta persona	picker ricercabile unico	<code>chips</code> · <code>select</code> · <code>legacy-id</code>

5.3 Mockup prima / dopo

A Campo del form — resa e stati

PRIMA • INPUT A MANO

Nome *

Email account (login)

Reparto

Asterisco solo su alcuni campi · nessun help · l'errore email **non compare** accanto al campo (solo flash in cima).

DOPO • RENDERER UNICO

NOME *

EMAIL ACCOUNT *

Inserisci un indirizzo valido (es. m.rossi@dominio.local).

REPARTO

Area e caporeparto vengono assegnati dal catalogo reparti.

Obbligatorio coerente · help contestuale · **errore accanto al campo** · focus visibile.

B Form di modifica — dal JS grezzo all'HTMX

PRIMA • CAMPI IN INNERHTML

```
// edit costruito come stringa
box.innerHTML =
  '<label>Descrizione *</label>' +
  '<textarea>' + descrizione + '</textarea>';
// label senza for · nessun errore
// server · dato grezzo → markup rotto
```

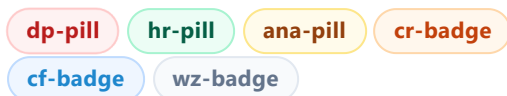
Divergente dalla create · perde help/errori · rischio escaping su <textarea>.

DOPO • WIDGET + HTMX INLINE

```
<!-- stessa riga, edit inline -->
<tr hx-get="/.../edit/"
    hx-swap="outerHTML">
  {% include "_field.html" %}
  <!-- validazione al blur -->
  hx-post hx-target="closest .nx-field"
</tr>
```

Stesso renderer della create · errori e help nativi · nessuna stringa HTML lato client.

C Stato: da 5 semafori a 1



Prima: 6 sistemi, verdi che non coincidono.



Dopo: un'unica scala fm-pill-*, semantica chiara.

D Scelta persona: da 3 a 1

Prima

- chips + picker con data-label
- `<select>` piatta senza ricerca
- *input numerico* "legacy id" digitato a mano

Dopo

DIPENDENTE *

Rossi Mario · CNO 0001

Rossini Ada · CNO 0184

Un unico picker ricercabile per tutti i contesti.

5.4 Velocità d'inserimento

- **Autofill dell'assist** (già in corso_form): codice dal titolo, validità dalla qualifica — da estendere.
- **Default intelligenti** e sezioni facoltative rinviabili (la progressività della create è già buona).
- **Validazione inline al blur** via HTMX: l'utente corregge subito, non dopo il submit.
- **Picker ricercabili** al posto di select lunghe e "tieni premuto Ctrl".
- **Edit inline** di riga senza ricaricare l'intera pagina (presenze, iscritti).
- **Tastiera**: invio per confermare, ESC per chiudere, focus ordinato.

5.5 Accessibilità & chiarezza degli errori — la specifica

REGOLA	COSA CAMBIA
Label associata	Ogni controllo ha <code>id</code> e <code>label for</code> ; il click sulla label focalizza il campo.
Errore doppio	Riepilogo in cima (link ai campi) e messaggio inline con <code>aria-invalid</code> / <code>aria-describedby</code> .
Obbligatorio	Un solo marcatore <code>.nx-req</code> + <code>aria-required</code> ; niente più 4 varianti.
Bottoni veri	Gli header collassabili diventano <code><button aria-expanded></code> , non <code><div onclick></code> .
Focus visibile	Anello di focus (già in <code>fmd</code>) applicato a tutti i controlli e alle azioni.
Conferme	Le azioni distruttive usano la modale di conferma coerente, non <code>confirm()</code> nudo.
Dark-mode	Solo token: sparisce il light-only degli hex inline e i bug di leak dark→light.

06 Alternative a confronto

Dal consolidamento SSR alle isole reattive — con raccomandazione

Il vincolo di riferimento è **Django SSR + HTMX**. Restando lì si risolve la quasi totalità dei debiti. Per completezza sono comparate anche le opzioni più spinte, con costo e rischio.

OPZIONE	SINTESI	COSTO	RISCHIO
A • Design system SSR + HTMX Raccomandata	Estrai <code>fmd-</code> in <code>core</code> , un renderer/pill/modale unici, validazione inline HTMX. Zero nuove dipendenze.	 medio	 basso
B • + Alpine.js (isole)	Solo per 3 primitive interattive (modale, picker ricercabile, collapse) in modo dichiarativo. ~15KB, no build. Compone con HTMX.	 medio-basso	 basso-medio
C • Web Components	I widget condivisi (picker, modale, pill) come custom element framework-agnostici. Progressivi, ma introducono tooling.	 medio-alto	 medio
D • Isole React	Solo per superfici davvero "app" (matrice editabile, widget drag&drop). Build pipeline, hydration, contraria alla direzione SSR.	 alto	 alto

RACCOMANDAZIONE

A come spina dorsale: risolve coerenza, errori e la maggior parte della velocità senza nuove dipendenze e in linea con `CLAUDE.md`. Adottare **B (Alpine) in modo selettivo** solo per le 3 primitive interattive, così da **eliminare pulitamente** il debito delle modali `innerHTML` e delle select senza ricerca. Tenere **D fuori portata**, salvo eventualmente la sola matrice competenze editabile in futuro. C resta un'opzione se in seguito si vuole condividere i widget con altri moduli del portale.

✓ Perché A + B selettivo

- ✓ Coerente con il vincolo e con il resto del portale.
- ✓ Riusa un sistema già maturo e testato (`fmd-`).
- ✓ Incrementale: si migra pagina per pagina, senza big-bang.
- ✓ Alpine tocca solo 3 widget, non l'architettura.

! Rischi da presidiare

- Disciplina: vietare nuovi namespace, code-review sul design system.
- Migrazione lunga: serve una roadmap a fasi (sezione 07).
- Alpine + HTMX: definire chi possiede lo stato per evitare doppioni.

07 Piano a fasi

Incrementale, con valore consegnato a ogni fase — nessun big-bang

F0	Estrazione del design system in core fondamenta Sposta i token e il sistema fmd- in core come base del portale; crea il renderer di campo unico <code>_field.html</code> e la mappa <code>fm-pill-*</code> canonica. Nessun cambiamento visibile: prepara il terreno.
F1	Quick win & bug fix alto impatto • basso costo Chiude i bug della sez. 3.3: <code>hr-btn-danger</code> , bottoni verbale <code>_dpi</code> , leak <code>dark→light</code> , paginatori che perdono i filtri, argomenti JS non quotati, <code>?v=</code> del CSS, doppia navigazione. Effetto immediato.
F2	Form ad alto traffico sul renderer unico velocità + errori Porta su widget + <code>_field.html</code> : creazione/scheda dipendente (sezione account), iscritti, presenze, wizard visite mediche. Recupera help, obbligatorio ed errori inline dove servono di più.
F3	Modale unica + picker ricercabile Alpine selettivo Sostituisce le ~10 modali inline con il componente unico e introduce il picker persona ricercabile — eliminando <code>innerHTML</code> , <code>select</code> senza ricerca e <code>input "legacy id"</code> a mano.
F4	Validazione inline HTMX + sweep accessibilità a11y + dark Validazione al blur, riepilogo errori, <code>label/for</code> , <code>aria-expanded</code> , conferme via modale, completamento <code>dark-mode</code> via token su tutte le pagine.
F5	Dismissione dei namespace legacy chiusura del debito Rimuove <code>dc/dp/ana/dr/qs/ro/mc/rf/cr/cf/wz/vm/vd...</code> e consolida le liste (una tabella dipendenti, un solo sorter). Da qui in avanti: un solo sistema da mantenere.

REGOLA D'ORO DELLA MIGRAZIONE

Ogni fase è **rilasciabile da sola** e non blocca le altre. F1 può partire subito; F2 dà il ritorno più visibile agli utenti; F5 si chiude quando l'ultimo namespace legacy sparisce.

08 Come misuriamo il successo

Obiettivi verificabili, allineati alle tre priorità

METRICA	OGGI	OBIETTIVO	PRIORITÀ
Namespace CSS attivi nel modulo	~20	1	Coerenza
Bottone: implementazioni distinte	7+	1	Coerenza
Form con errori mostrati accanto al campo	≈0%	100%	Errori
Marcatore obbligatorio: varianti	4+	1	Accessib.
Modi per "scegli dipendente"	3	1	Velocità
Modali con ESC / focus-trap	1	tutte	Accessib.
Violazioni accessibilità (axe) sui form chiave	molte	0	Accessib.
Pagine con dark-mode completo	parziale	tutte	Coerenza
Bug della sez. 3.3 aperti	8	0	Robustezza

→ Prossimo passo consigliato

- Validare questo brief e la scelta **A + B selettivo**.
- Aprire **F1** (quick win) come primo incremento, subito visibile.
- In parallelo, **F0**: estrarre il design system in core.
- Definire i primi form di **F2** (creazione dipendente + iscritti).

◇ In una riga

Un modulo che si presenta come **un solo prodotto**: form più veloci, errori chiari dove servono, una libreria sola da mantenere — **a parità di stack** e per incrementi rilasciabili.